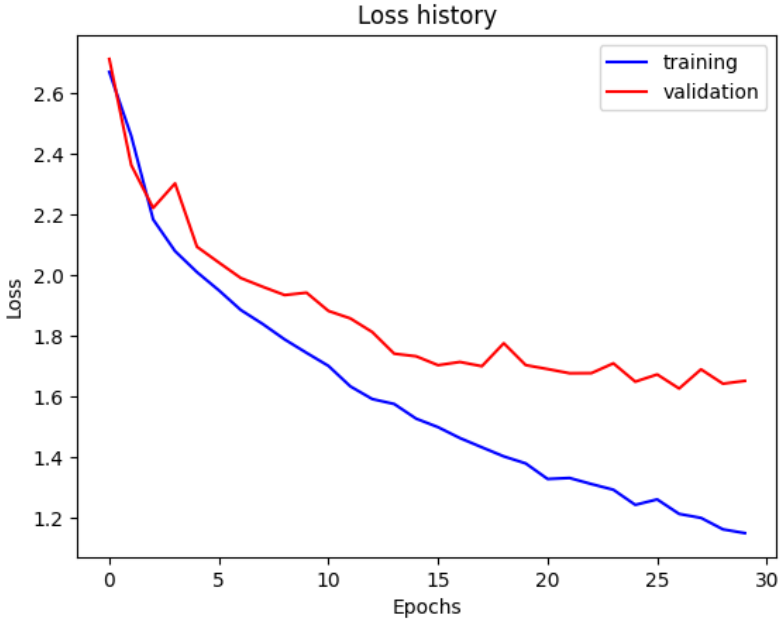
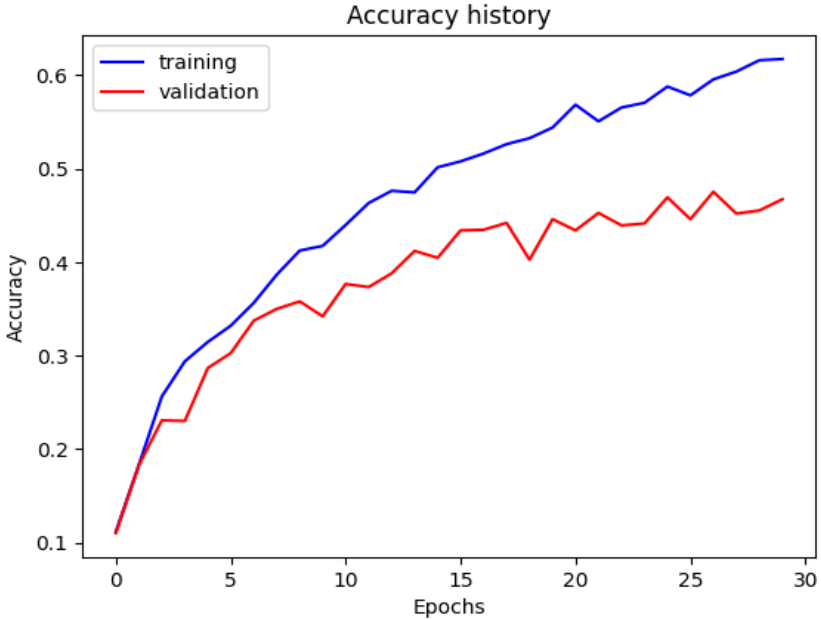
CS 6476 Project 3

Shruti Jeyaraman
sjeyaraman6@gatech.edu
sjeyaraman6
904143748

Part 1: SimpleNet





Final training accuracy: 61.74%

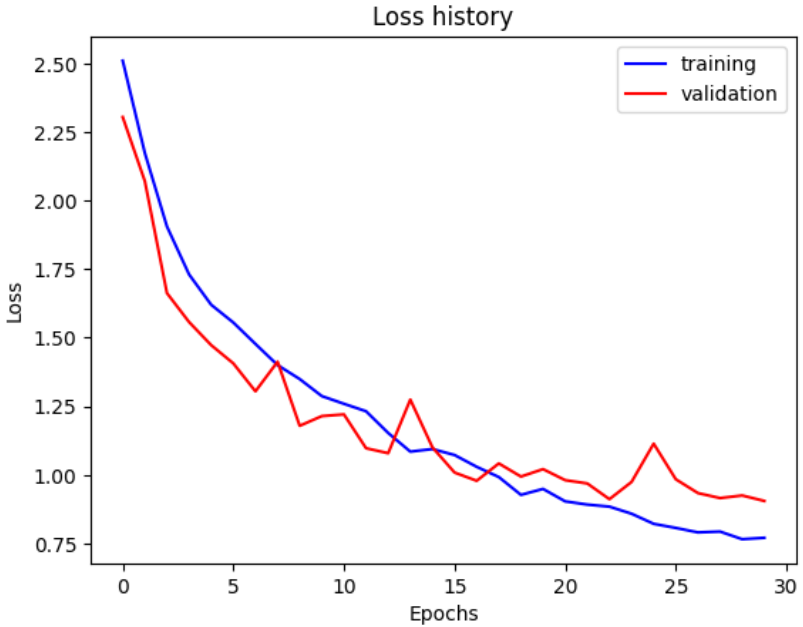
Final validation accuracy: 46.73%

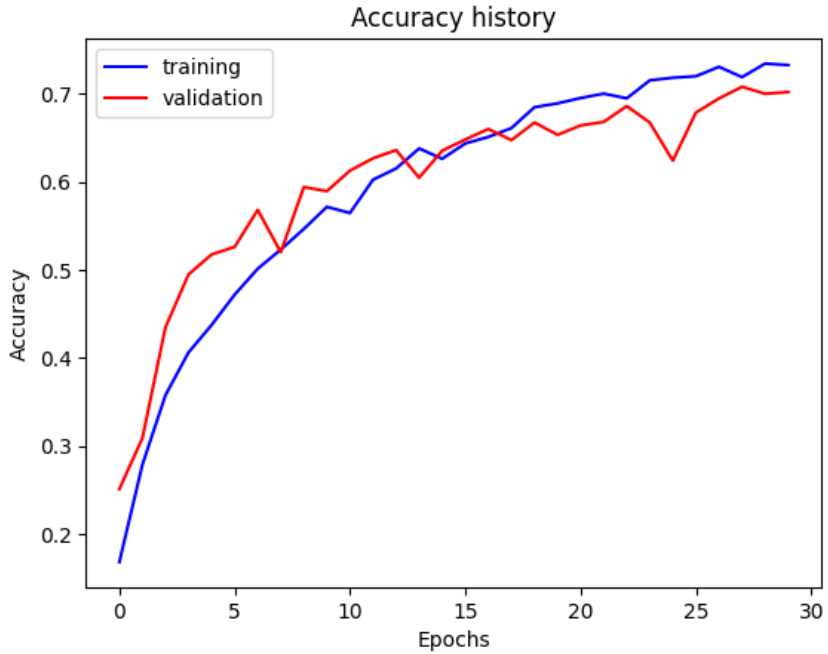
Part 2: SimpleNetFinal

Add each of the following (keeping the changes as you move to the next row):

	Training accuracy	Validation accuracy
SimpleNet	61.74%	46.73%
+ Jittering	54.41%	50.33%
+ Zero-centering & variance-normalization	77.82%	58.87%
+ Dropout regularization	64.19%	56.80%
+ Making network "deep"	63.69%	61.87%
+ Batch normalization	73.27%	70.2%

Part 2: SimpleNetFinal





Final training accuracy: 73.27%

Final validation accuracy: 70.20%

Part 2: SimpleNetFinal

Name 10 different possible transformations for data augmentation:

1. Horizontal flipping – image flipping about the vertical axis
2. Vertical flipping – image flipping about the horizontal axis
3. Random rotation – image rotation anti-/clockwise by a specific angle to handle different tilts
4. Zooming in and out
5. Translation
6. Shearing – applying an affine distortion that skews the angle of the image along one axis
7. Color jitter – Modifying the saturation, hue or individual RGB channels to prevent the model from relying on specific color tints
8. Gaussian Noise Injection – Adding random pixel-level static to make the model resilient to low-quality or grainy sensors
9. Gaussian Blurring / Filtering — Smooths pixel values via weighted local averaging to simulate out-of-focus or motion-blurred inputs
10. CutMix – Patching a section of one training image onto another while proportionally blending their target labels

What is the desired variance after each layer? Why would that be helpful?

We want the activations and gradients at every layer of the network to be useful for learning so ideally, we want the variance of the inputs after each layer to be 1. This way the activations maintain the same scale across layers without exploding or shrinking to the point that the model stops learning. Without normalization, the scale of every layers' inputs drift as earlier layers update. Normalization ensures training is smoother and that the layers actually learn from the data. It also means that the optimizer can increase learning rates without diverging and converge faster.

Part 2: SimpleNetFinal

What distribution is dropout usually sampled from?

Dropout samples from a Bernoulli distribution

$$f(x) = \begin{cases} p^x * (1-p)^{1-x} & if\ x = 0,1 \\ 0 & otherwise \end{cases} = \begin{cases} p & if\ x = 1 \\ 1-p & if\ x = 0 \end{cases}$$

In the forward pass, neurons are retained with probability (1-p) and dropped with probability p, which for our code is set to 0.5.

Pytorch uses inverted dropout where the retained neurons are scaled by 1/(1-p) so the expected value of each activation stays the same during training.

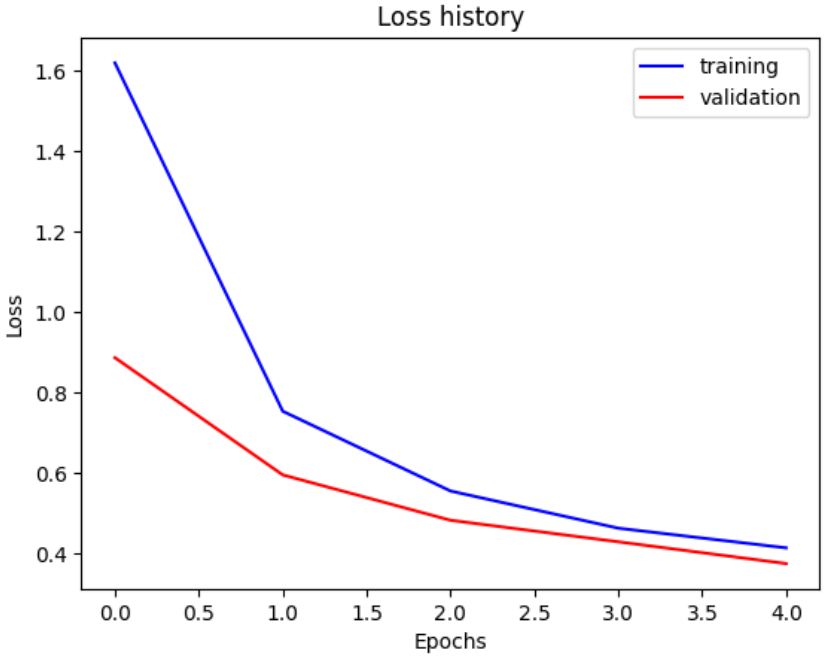
How many parameters does your base SimpleNet model have? How many parameters does your SimpleNetFinal model have?

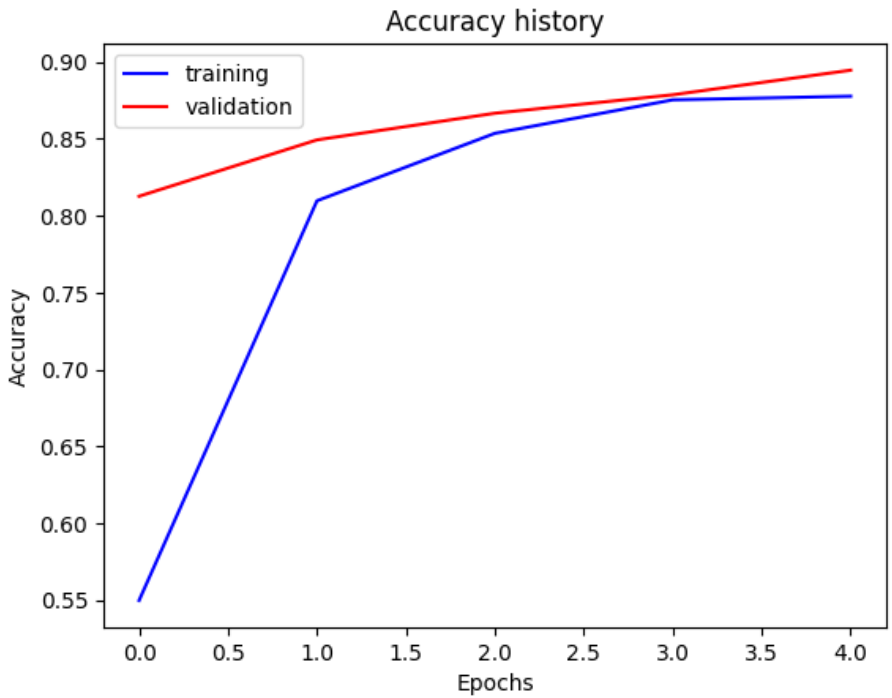
My base SimpleNet model has 56,895 parameters. SimpleNetFinal has 361,583 parameters

What is the effect of batch norm after a conv layer with a bias?

Convolution with bias simply aadds a constant per output channel y = Wx=b. Batch norm subtracts each channel's mean over the batch. For the entire channel, bias b is constant everywhere within so the b gets removed entirely during batchnorm. The output doesn't depend on b so its gradient is zero and the layer never learns anything. BatchNorm's own learnable shift B applies after normalization and shifts the output.

Part 3: ResNet

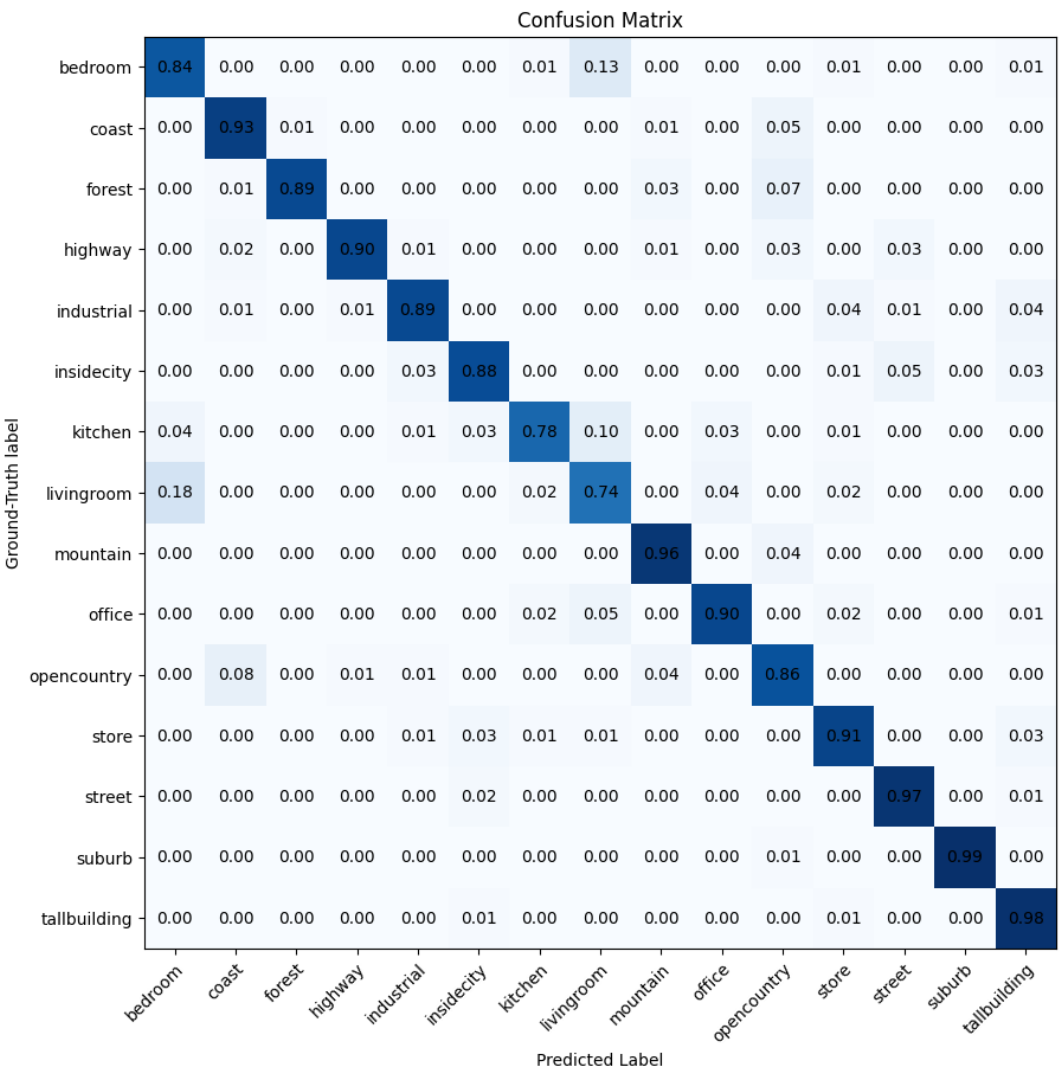




Final training accuracy: 87.77%

Final validation accuracy: 89.47%

Part 3: ResNet



Part 3: ResNet

Insert visualizations of 3 misclassified images from the most misclassified class according to your confusion matrix. Explain why this may have occurred.

True: livingroom
Predicted: bedroom
image_0006.jpg



Model is probably confused by the fact that this sofa in the bottom right of the image looks like a bed

True: livingroom
Predicted: kitchen
image_0038.jpg



This specific shot of the living room has more of the kitchen itself in the scene which lead to the misclassification

True: livingroom
Predicted: office
image_0051.jpg



I believe the misclassification here might be occurring with the look of the center table and the arm rests and legs of the couch which look more like office furniture

Part 3: ResNet

Fine-tuning is the specific task of modifying or updating a subset of weights of a pretrained neural network or foundational model with more generalized capabilities, towards performing a specific task or achieving a specific goal. This is done by training this largr “frozen” model on task-specific data to update those correspondig weights. Instead of building and training a network completely from scratch—which requires massive data and high computing costs—fine-tuning builds upon existing knowledge. This is great for saving on computational complexity, improving domain-specific accuracy and works with a much smaller pool of data.

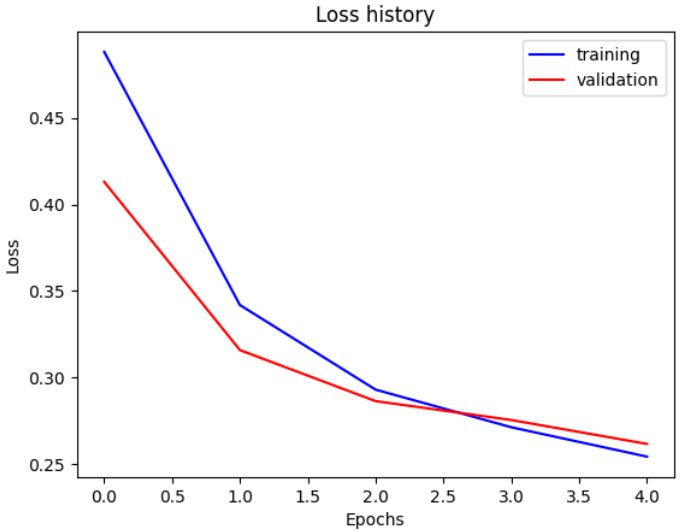
Why do we want to "freeze" the conv layers and some of the linear layers from a pre-trained ResNet? Why can we do this?

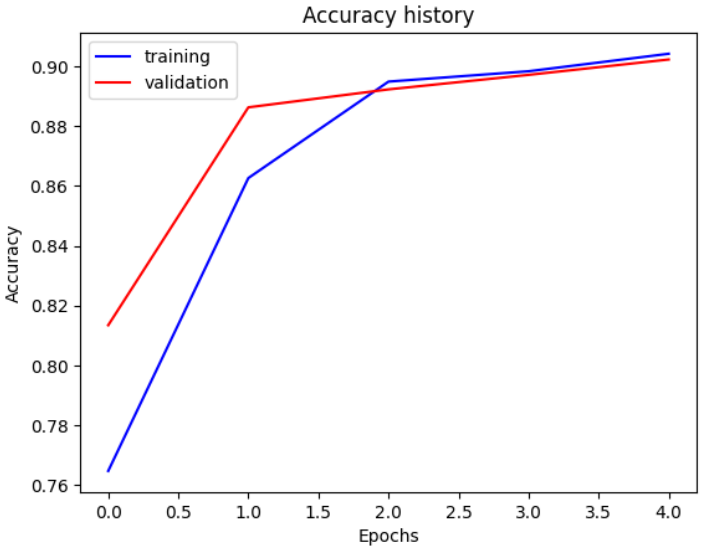
Freezing the conv layers and linear layers in the pretrained ResNet is important to reduce the risk of overfitting when we train our ~11M parameter model, with our data of only 2985 training images. This also retains the overall knowledge of the larger network and allows us to train faster and cheaper and keep learning more reliable. We are able to do this because

a. The earlier layers have a smaller receptive field so they learn general-purpose features like edges, textures and shapes
b. Our scenes in the training set are natural photos like ImageNet’s so the original ResNet’s features are already separating the scene types well enough
c. The last classifying layer is the only thing that needs weight update specific to the training data

The tradeoff is that the frozen features cant adapt to the differences in our training data. ResNet was traind on color images and our training images are grayscale copied into 3 channels. Our training images are also indoor scene images whie ImageNet had object-focused images.

Part 4: Multi-label Scene Attributes (Extra Credit)



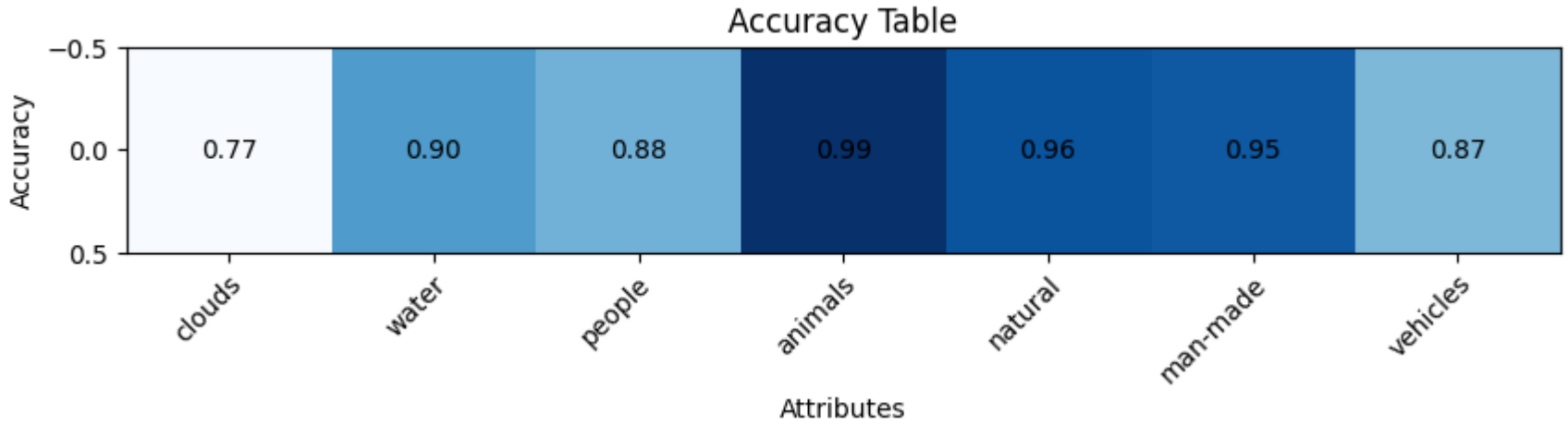


Final training accuracy: 90.42%

Final validation accuracy: 90.23%

Part 4: Multi-label Scene Attributes (Extra Credit)

[Insert visualization of accuracy table obtained from your final MultilabelResNet model.]



Part 4: Multi-label Scene Attributes (Extra Credit)

List 3 changes that you made in the network compared to the one in part 3.

The same pretrained ResNet 18 with all its layers is used for this part with the following three changes:

a. The output size has gone from 15 to 7. The final linear layer produces one ouptut for each of the seven attributes instead of just one per scene class
b. A sigmoid has also been added after the final layer so the class probabilities lie within the expected [0,1] range for the detection of specific attributes which is anchored on a robability>0.5 rule
c. We switched from cross entropy loss to binary cross entropy loss averaged over all 7 attributes

[Is the loss function of the ResNet model from part 3 appropriate for this problem? Why or why not?]

Cross entropy loss applies a softmax across the outputs, so the predicted probabilities add up to 1. That assumes exactly one correct class per image. In Part 4, an image can have several attributes at once, or none. A street scene is often man-made, has vehicles and has people. Under softmax, raising the probability for "people" forces the others down, so the model can't say yes to all three. Cross-entropy also expects one class index per image as the target, while each label here is a vector of 7 zeros and ones.

Sigmoid with BCE treats each attribute as its own yes/no question, which matches the problem.

Part 4: Multi-label Scene Attributes (Extra Credit)

Explain a problem that one needs to be wary of with multilabel classification. HINT: consider the purpose of visualizing your results with the accuracy table. You might want to do some data exploration here.

A key pitfall in multilabel classification is class imbalance, which makes overall accuracy misleading. Exploring the attribute labels for the five scene classes shows how skewed they are. Animals appear in only 1.2% of test images (2.6% of training images), and people appear in 16.6%. A model that always predicts "no animals" would score 99% on that attribute without ever detecting one, and always predicting "no people" would score 88%. Averaged over all seven attributes, always guessing the more common answer already reaches about 69% accuracy without looking at the images. Because the trainer's accuracy averages over every image–attribute pair, a high overall number like our 90%+ can hide a model that simply ignores rare attributes. This is why the per-attribute accuracy table is useful: each attribute's accuracy should be compared against its own baseline, and for rare attributes, precision and recall (or F1) are more informative than accuracy.

The labels also pose a second problem: they are strongly tied to scene type and shift between splits. Highway and street images are 100% man-made and 0% natural, every coast image contains water, and roughly 90% of highway and street images contain vehicles. The model can therefore score well by recognizing the scene category and inferring its typical attributes, rather than detecting each attribute in the image. The positive rates also differ between training and test: natural drops from 69.6% to 57.8% and vehicles rise from 31.7% to 43.2%. That can make the model favor answers that are common in training but less so at test time. Remedies include weighting positive examples more heavily in the loss, tuning a separate decision threshold for each attribute, and reporting per-attribute metrics instead of a single overall accuracy.